

The Worktrees tab

What the `/worktrees` page of the orchestrator web UI lists, where each worktree comes from, and exactly what archiving one does to the folder, the branch and the database.

Repository: `/Users/hazem/orchestrator` (branch `docs/worktrees-tab-report`, commit `ff65a12`).

Every claim cites `file:line`. Anything the code does not state directly is marked (*inferred*).

Paths under `apps/web/src/` are written `web/...`; under `apps/daemon/src/` as `daemon/...`; under `packages/` as `pkg/...`.

Contents

- 0. The short answer: is archiving deleting?
- 1. What the user sees
- 2. Where worktrees come from: app-created and discovered
- 3. Archiving, traced step by step
- 4. Auto-archive on merge and the Inbox rows it raises
- 5. Live data from the running daemon
- 6. Declarations
- 7. What the code does not settle

0. The short answer

No. Archiving deletes the worktree folder from disk, but keeps the branch, its commits and the database row.

- **Folder: deleted.** The daemon runs `git worktree remove <path>` without `--force`.
[daemon/services/worktree/worktree-sync.ts:105; daemon/services/git/exec.ts:45-46]
- **Uncommitted work: never lost.** Archive refuses with `dirty_worktree` when `git status --porcelain=v1 --untracked-files=all` lists any file (modified, staged or untracked). [worktree-sync.ts:95-101; worktree-read.ts:57-61]
- **Branch: kept, local and remote.** No `git branch -d` and no push runs. The git wrapper refuses branch deletion and `push --delete` outright. The confirm dialog text says "Branch <name> is kept."
[exec.ts:26-27, 40, 49-50; daemon/http/routes/worktrees.ts:133]
- **Database row: kept.** It is set to `state='archived'`, `archivedAt=now`, `dirty=false`.
[daemon/db/repos/worktrees.ts:78-83]

The nuance – three things are deleted besides the folder's tracked files:

- Files git ignores (for example `node_modules`, or `.env` files copied in by `copyGlobs`). They do not count as dirty, and `git worktree remove` deletes the whole folder. (*inferred: git behaviour, not repo code*) [worktree-read.ts:58; worktree-write.ts:32-61]
- The backup refs under `refs/orchestrator/reverts/<key>/` that the Diff "revert" action writes before it reverts a file. [worktree-sync.ts:65-71, 94, 109; daemon/services/diff/diff.ts:21-25, 99]
- Whatever the repo's optional `archive` script removes. It runs inside the folder first, with a 300 s timeout. [worktree-sync.ts:102-104; pkg/api-contract/src/config.ts:25]

Undo: there is no restore button and no restore endpoint. Because the branch survives, `git worktree add <path> <branch>` followed by **Discover** brings the same row back to `active`, and it keeps its old `createdByApp` value. [worktree-read.ts:218-248]

External worktrees (not created by the app) archive only with an explicit extra confirmation, and `auto-archive` never archives them. [worktree-sync.ts:88-93; routes/worktrees.ts:136-144; auto-archive.ts:114-117]

1. What the user sees

Orchestrator

Wakecap (1049)

Search sessions...

Search... 5h \$0.00 7d \$10776 \$0.00/h estimated

Inbox 9

New session

System

Inbox

Live

Worktrees

History

Settings

Audit

Streams

Analytics

Automations

Worktrees

Discover

New worktree

Branch	Ticket	State	PR	Sessions	
/Users/hazem/Wakecap/Backend/identity-service					
master	—	main checkout	—	3	</> ☐
/Users/hazem/Wakecap/Backend/infrastructure					
master	—	main checkout	—	7	</> ☐
chore/drop-dead-observationfilters-config	—	external	#5924 · merged	0	</> ☐ ...
chore/bump-stage-apps-module-v2.3.0	—	external	#5178 · closed	0	</> ☐ ...
docs/ALU-1293-wecare-stage-pin-caveat	ALU-1293	external	#5173 · merged	0	</> ☐ ...
feat/ALU-1293-obs-failed-routes	ALU-1293	external	#5333 · merged	0	</> ☐ ...
feat/ALU-1293-obs-status-totals	ALU-1293	external	#5268 · merged	0	</> ☐ ...
feat/ALU-1293-observation-alerts	ALU-1293	external	#5085 · merged	0	</> ☐ ...
feat/ALU-1293-observation-alerts-prod	ALU-1293	external	#5216 · merged	0	</> ☐ ...
feat/ALU-1293-observation-alerts-stage	ALU-1293	external	#5101 · merged	0	</> ☐ ...
feat/ALU-1293-observation-metrics-prod	ALU-1293	external	#5183 · merged	0	</> ☐ ...
feat/ALU-1293-observation-metrics-stage	ALU-1293	external	#5099 · merged	0	</> ☐ ...
feat/ALU-1293-observation-paging-route	ALU-1293	external	#5261 · closed	0	</> ☐ ...
feat/ALU-1293-observation-slack-route	ALU-1293	external	#5094 · merged	0	</> ☐ ...
feat/ALU-1293-wecare-alerts	ALU-1293	external	#5170 · merged	0	</> ☐ ...
feat/ALU-1293-wecare-dashboard	ALU-1293	external	#5165 · merged	0	</> ☐ ...
feat/ALU-1293-wecare-metrics-plumbing	ALU-1293	external	#5159 · merged	0	</> ☐ ...

The Worktrees tab at `http://localhost:5173/worktrees`, 1440×1000, headless Chrome against the running dev server on 2026-09-30. The project selector is on Wakecap, so the list is scoped with `projectId`. Every non-main row here is `external`, and most carry a merged PR.

Route and shell

- `/worktrees` renders `<WorktreesPage />`. [web/routes/worktrees.tsx:4–8]
- "Worktrees" is the third link in the desktop left nav, after Inbox and Live. [web/features/shell/AppShell.tsx:160–166]

What it lists

- Only **active** rows. The page always sends `state: 'active'`, plus `projectId` when a project is selected. [web/features/worktrees/WorktreesPage.tsx:50–51]
- The page has no archived view and no filter controls of its own. The only filter is the global project selector. The API also accepts `state=archived` and `repo`, but the page never sends them. [pkg/api-contract/src/routes/worktrees.ts:26–30]
- Rows include each repo's **main checkout** (`isMain`), shown with a "main checkout" badge and no "..." menu. [web/features/worktrees/WorktreeRow.tsx:40, 98]
- Rows are grouped by `repo` under a heading row holding the repo path. The daemon orders by `repo`, then `path`, ascending. [WorktreesPage.tsx:71–75, 167–178; daemon/db/repos/worktrees.ts:73]

Columns

Column	Shows	Source field and rule
Branch	Branch name in mono. <code>(detached)</code> for a detached HEAD.	<code>branch</code> [WorktreeRow.tsx:136; daemon/services/worktree/discover.ts:81]
Ticket	Outline badge, or an em-dash.	<code>ticket</code> , derived from the branch name (§2.3). [WorktreeRow.tsx:137–145]
State	One of <code>main checkout</code> / <code>app</code> / <code>external</code> , plus a red <code>dirty</code> badge when there are uncommitted files.	<code>isMain</code> , <code>createdByApp</code> , <code>dirty</code> [WorktreeRow.tsx:37–47]
PR	Link text <code>#N · merged</code> , <code>#N · closed</code> , or for an open PR <code>#N · checks passing failing running</code> / <code>no checks</code> . Plain <code>PR</code> when a URL exists without cached status. Em-dash otherwise.	<code>prStatus</code> , else <code>prUrl</code> [WorktreeRow.tsx:16–28, 49–59]
Sessions	A count.	<code>sessionPks.length</code> : sessions from the last 30 days whose working directory is inside this worktree. [WorktreeRow.tsx:154; worktree-read.ts:74–88]
(actions)	VS Code and Terminal icons; "..." menu with Run, Sync to main checkout, Archive...	See the actions table. [WorktreeRow.tsx:79–131]

Actions and the API calls behind them

UI action	HTTP call	What the daemon does
Discover (header button)	<code>POST /api/worktrees/discover</code>	Runs discovery now (§2.2), returns the active list; the web invalidates every <code>['worktrees']</code> query. [WorktreesPage.tsx:102–105; web/api/queries/worktrees.ts:19–25; routes/worktrees.ts:39–42]
New worktree	<code>POST /api/worktrees</code>	Confirm step, then <code>createWith</code> ; optionally launches Claude in the new folder (§2.1). [web/features/worktrees/CreateWorktreeDialog.tsx:76–85; routes/worktrees.ts:51–91]
VS Code icon	<code>POST /api/worktrees/open</code> <code>target:'vscode'</code>	<code>code --new-window <path></code> , no confirm. [WorktreesPage.tsx:79–82; worktree-write.ts:154–166]
Terminal icon	same, <code>target:'terminal'</code>	<code>open -a Terminal <path></code> . [worktree-write.ts:164]
Run	<code>POST /api/worktrees/script</code> <code>which:'run'</code>	Confirm dialog shows the script, then the repo's <code>run</code> script starts in a PTY (pseudo-terminal) and opens in the terminal drawer. [WorktreesPage.tsx:65–69; routes/worktrees.ts:93–101; worktree-write.ts:134–148]
Sync to main checkout	<code>POST /api/worktrees/sync</code>	Confirm dialog lists every changed file; then copies them into the main checkout. Refused with <code>main_dirty</code> if the main checkout has local edits to the same files. [routes/worktrees.ts:114–124; worktree-sync.ts:27–63]
Archive...	<code>POST /api/worktrees/archive</code>	Two-step confirm, then <code>archiveAs(path,'user',...)</code> (§3). [WorktreesPage.tsx:57–61, 89–91, 185–193; routes/worktrees.ts:126–146]

- **Confirm pattern.** Every guarded action is first sent with `confirm:false`. The daemon answers `409 confirmation_required` with a summary; the page shows it in `GitConfirmDialog` and resends with `confirm:true`. [web/features/git/useConfirmedMutation.ts:16-79; daemon/http/routes/git-guard.ts:39-45]
- **Archive of an external worktree.** The dialog shows a checkbox "I understand this worktree was created outside the app"; the Archive button stays disabled until it is ticked, and the resend adds `confirmExternal:true`. [web/features/git/GitConfirmDialog.tsx:36, 49-54, 70-75]
- The API also supports `target:'finder'` and scripts `setup` / `archive`; the page does not expose them. [pkg/api-contract/src/routes/worktrees.ts:50-58; WorktreesPage.tsx:65-67, 77-93]

Live updates, phone layout, loading and error

- **Live patch.** A `worktree.updated` WebSocket frame replaces or appends the row in every cached `['worktrees', ...]` list; `worktree.removed` drops it. [web/api/live-events.ts:54-77]
- **Phone** (below 768 px): each worktree is a card with the same badges, ticket, PR and "N sessions"; empty fields are left out. [WorktreeRow.tsx:162-186; WorktreesPage.tsx:138-150]
- **Empty:** "No worktrees yet" with a Discover button. **Loading:** four skeleton rows with `aria-label="Loading worktrees"`. **Error:** a destructive alert "Couldn't run that action" carrying the first error from archive, sync, script or the list query. [WorktreesPage.tsx:30-47, 95, 111-137]

2. Where worktrees come from

A row enters the `worktrees` table in exactly two ways: the app creates the folder itself (`createWorktree`, `createdByApp=true`), or discovery finds a folder git already knows about (`discoverWorktrees`, `createdByApp=false` unless an earlier row said otherwise). [worktree-write.ts:102-121; worktree-read.ts:224-242]

2.1 App-created worktrees

Every app flow calls `createWith`, which runs `createWorktree`. [daemon/services/worktree/worktree.ts:65-66]

Flow	Where it starts	Setup script · actor	Branch slug
Worktrees tab → New worktree	<code>CreateWorktreeDialog</code> → <code>POST /api/worktrees</code> [CreateWorktreeDialog.tsx:76-85; routes/worktrees.ts:61-63]	checkbox (default on) · <code>user</code>	User's type / ticket / short description
New worktree + "Launch Claude in the new worktree"	same route with <code>launch</code> ; it builds a launch request with a <code>worktree</code> field [routes/worktrees.ts:66-90]	always on; the dialog's checkbox is ignored [launch-prepare.ts:49] · <code>user</code>	same
New session dialog → "New worktree for this ticket"	<code>LaunchPhase4Fields</code> → <code>launch</code> → <code>prepareLaunch</code> [web/features/launch/LaunchPhase4Fields.tsx:55-63; daemon/services/launch-prepare.ts:32-60; launch.ts:171]	on, waits up to 10 min · <code>user</code>	User's slug
Compare (one worktree per variant)	<code>compare.ts</code> <code>launch</code> [daemon/services/compare/compare.ts:137-146]	on · <code>user</code>	<code><slug>-v<n>-<source>[-<model>]</code>
Automations with <code>useWorktree</code>	<code>prepareCwd</code> [daemon/services/automations/service.ts:255-269]	off · <code>automation</code>	<code>auto-<name>-<runId[0..6]></code> , type <code>fix</code> for GitHub triggers else <code>chore</code>

- **Ship does not create worktrees.** After `gh pr create` it writes the new PR URL onto the worktree that contains the checkout. [daemon/services/ship/ship.ts:161-167]

What `createWorktree` does

1. **Branch name** = `<type>/<TICKET>-<slug>`, or `<type>/<slug>` without a ticket. The slug is lower-cased, filler words dropped, max 30 chars. [pkg/core/src/git/branch.ts:7-32]
2. **Repo check.** `repo` must be the main checkout, or it fails with `not_a_worktree`. [worktree-write.ts:77-80]
3. **Path** = `<repo>/<worktreeDir>/<branch with "/" → "-">`; `worktreeDir` defaults to `.worktrees`. [worktree-write.ts:81-83; branch.ts:34-36; config.ts:27]
4. **Clash check.** Fails with `worktree_exists` if git already has a worktree on that branch or path, or the folder exists. [worktree-write.ts:85-89]
5. **git.** Existing local branch → `git worktree add <path> <branch>`; otherwise `git worktree add -b <branch> <path> <base>`. [worktree-write.ts:91-96]
6. **Copy ignored files** from the main checkout that match the repo's `copyGlobs`, never overwriting. [worktree-write.ts:32-61, 99]

7. **Row:** `createdByApp=true`, `origin='app'`, `state='active'`, `base` set, ticket upper-cased, `createdAt` kept if the path had a row before. Emits `worktree.updated`. [worktree-write.ts:100-123]
8. **Setup:** when asked and configured, the repo's `setup` script runs in a PTY. [worktree-write.ts:125-128]
9. The whole call is recorded in the audit log as `worktree.create`. [worktree-write.ts:70-76]

2.2 Discovery (`discoverWorktrees`)

- one step, naming its file tinted band = one stage of the pipeline
 dashed = runs only when a condition holds

1 • Collect candidate folders

`collectCandidates` – daemon/services/worktree/sources.ts:89-138

config

each `projects[].repos[].path`
(:99-100)

worktree-dir

every entry of
`<repo>/<worktreeDir>` (:102-106)

sibling

folders next to a configured repo whose `.git` is a file; when `scanSiblings` (default true) (:108-117; config.ts:214)

scratchpad

folders named `claude-*` under `scratchpadRoots` (default `/private/tmp`), depth ≤ 6 (:119-121; config.ts:213)

claude-json

`githubRepoPaths` from `~/claude.json`, nothing else read (:18-32, 123-124)

session-cwd

working dirs of up to 300 sessions from the last 30 days (:126-136)

↓ candidate folders

2 • Ask git

`resolveWorktrees` – daemon/services/worktree/discover.ts:32-92

main checkout

each candidate → its main checkout via `git worktree list` (:45-51)

list worktrees

`git worktree list --porcelain` once per main checkout; bare entries skipped (:63-67)

drop stale

skip entries git marks prunable whose folder is gone (:77)

↓ one record per worktree: path, repo, branch, head, isMain, origin

3 • Derive fields and store

`discoverWorktrees` – daemon/services/worktree/worktree-read.ts:197-258

build row

ticket, dirty, PR link, sessions, project (\$2.3, \$2.4) (:216-243)

write if changed

upsert + `worktree.updated` only when a fingerprint of 10 fields differs (:183-195, 245-248)

mark vanished

an active row not found and whose folder is gone → `archived` + `worktree.removed` (:252-256)

- **Origin** of a non-main worktree is fixed by where it sits, in this order: inside `<main>/<worktreeDir>` → `worktree-dir`; under a scratchpad root → `scratchpad`; same parent folder as the main checkout → `sibling`; otherwise the candidate's own origin. [discover.ts:69-76] A row that was ever `createdByApp` keeps `origin='app'`. [worktree-read.ts:236]
- **External** means `createdByApp=false`. Discovery never sets it to true; it copies the previous row's value or defaults to false. [worktree-read.ts:233]

2.3 How each field is derived

Field	Rule
<code>branch</code>	From <code>git worktree list --porcelain</code> ; <code>(detached)</code> when none. [discover.ts:81]
<code>ticket</code>	<code>ticketFromBranch(branch, project.ticketRegex)</code> ; default regex <code>\b([A-Za-z][A-Za-z0-9]{1,9}-\d+)\b</code> , case-insensitive, upper-cased. Always null for the main checkout. [worktree-read.ts:220-221, 229; pkg/core/src/git/branch.ts:3, 38-45]
<code>projectId</code>	<code>ctx.projects.resolve(path)</code> : the project whose path prefix contains the worktree. [worktree-read.ts:220]
<code>dirty</code>	Any file in <code>git status --porcelain=v1 -z --untracked-files=all</code> . [worktree-read.ts:57-61, 230]
<code>sessionPks</code>	Sessions from the last 30 days (max 300) with a working dir inside the path; the deepest matching worktree wins. [worktree-read.ts:74-88]
<code>base</code>	Only set by <code>createWorktree</code> ; discovery keeps the previous value. [worktree-read.ts:228]
<code>head</code> , <code>isMain</code>	From git; <code>isMain</code> when the path equals the main checkout. [discover.ts:69, 82-83]
<code>prUrl</code>	<code>linkedPrUrl</code> – see §2.4.
<code>prStatus</code>	Not stored. <code>toView</code> reads the cached PR for <code>prUrl</code> , or, with no URL, the newest cached PR whose head branch equals this branch in the same repo. [worktree-read.ts:63-72; daemon/db/repos/pr-cache.ts:45-55]

2.4 The PR link (`linkedPrUrl`)

Own-repo rule: a worktree links only a PR from its own GitHub repo. The repo slug (`owner/name`) is read from `git remote get-url origin`. [worktree-read.ts:36-55, 94-100] Steps, first hit wins:

1. Main checkout → no link. [:108]
2. **Keep the stored link** if it is from the same repo (or the slug is unknown) and its cached PR's head branch is not a different branch. [:109-116]
3. **PR cache by head branch** in this repo: `findPrByHead(branch, slug)`. [:117-118]
4. No slug → keep a stored link or give none. Session PRs are never used without a slug. [:119]
5. **GitHub by head branch:** `gh pr list -R <slug> --head <branch> --state all --limit 1`, only when `github.enabled`, and at most once per path+branch per 10 minutes while the result stays stored. An open PR found this way is written to the cache. [:90-92, 121-139; daemon/connectors/github/github.ts:82-105]
6. **Session fallback:** the first PR recorded on a linked session that is in the same repo and whose cached head is not a different branch. [:137, 144-148]
7. If `gh` fails: keep a same-repo stored link, else the first same-repo session PR. [:131-134]

Ship also sets `prUrl` directly when it opens a PR. [ship.ts:161-167]

2.5 When discovery runs

- At daemon start and then every 5 minutes (`DISCOVER_EVERY_MS = 5 * 60_000`). [daemon/main.ts:40, 64-71]
- When the user clicks **Discover**. [routes/worktrees.ts:39-42]
- On every Streams refresh, which calls `discover()` to collect worktrees per ticket. [daemon/services/streams/streams.ts:139]
- Calls overlap safely: a second call while one runs gets the same promise. [worktree.ts:57-64]

3. Archiving, traced step by step

All three callers end in one function, `archiveWorktree(d, path, actor, {allowExternal})`, exposed as `archiveAs`. [daemon/services/worktree/worktree-sync.ts:73-115; worktree.ts:70-71]

Caller	actor	allowExternal	Source
Worktrees tab → Archive...	user	confirmExternal from the dialog	routes/worktrees.ts:126-146
Compare → archive losers	user	not passed → false	compare.ts:273-311; daemon/http/routes/compare.ts:36-52
Auto-archive on merge	automation	false	auto-archive.ts:118

 step that runs refusal: the archive stops, nothing is changed
 dashed = runs only when a condition holds

1 • Route guard (UI only)

row must be active, else 404.
First call returns 409
confirmation_required "Remove
worktree X. Branch Y is kept."
(routes/worktrees.ts:128-135)

external, not confirmed

409 external_worktree unless
confirmExternal
(routes/worktrees.ts:136-143)

not active / main checkout

not_a_worktree / is_main_checkout
(worktree-sync.ts:13-18, 87)

↓

2 • external check

!createdByApp && !allowExternal →
external_worktree (:88-93)

3 • dirty check

folder exists and any modified,
staged or untracked file →
dirty_worktree "uncommitted
changes in N file(s)" (:95-101)

4 • archive script

repo's archive script, \$SHELL -
lc, cwd = worktree, 300 s timeout
(:102-104)

↓

5 • remove folder

git worktree remove <path> run
from the main checkout; no --
force (:105)

5' • folder already gone

git worktree prune instead (:106-
108)

6 • delete revert refs

git update-ref -d for each ref
under
refs/orchestrator/reverts/<key>/
(:65-71, 94, 109)

↓

7 • mark row archived

state='archived', archivedAt,
updatedAt, dirty=false (:110-111;
db/repos/worktrees.ts:78-83)

8 • emit

worktree.removed → row leaves the
page; Inbox archive rows for this
path resolve (\$4) (:112; auto-
archive.ts:105)

9 • audit

one worktree.archive audit record
with actor, path and
allowExternal, success or error
(:80-86;
daemon/services/git/audit.ts:10-19)

What happens to each thing

Thing	Result	Why
Worktree folder	Deleted	git worktree remove. [worktree-sync.ts:105]
Uncommitted changes (modified, staged, untracked)	Archive refused	Dirty check before any change. [worktree-sync.ts:95-101] Forced removal is also blocked in the git wrapper. [exec.ts:45-46]

Thing	Result	Why
Git-ignored files inside the folder	Deleted	Not reported by the dirty check; removed with the folder. <i>(inferred: git behaviour)</i>
Local branch and its commits	Kept	No branch command runs; <code>branch -d/-D/--delete</code> is refused by <code>assertSafeGitArgs</code> . [exec.ts:49-50]
Remote branch	Kept	No push runs; <code>push --delete</code> and <code>push :ref</code> are refused. [exec.ts:20-42]
Revert backup refs	Deleted	<code>pruneRevertRefs</code> . [worktree-sync.ts:65-71, 109]
DB row	Kept, archived	<code>markWorktreeArchived</code> ; still returned by <code>GET /api/worktrees?state=archived</code> . [db/repos/worktrees.ts:61-83]
Linked sessions	Unchanged	Archive does not touch sessions. Compare kills a loser's PTY itself before archiving. [compare.ts:282-301]

The other way a row becomes archived

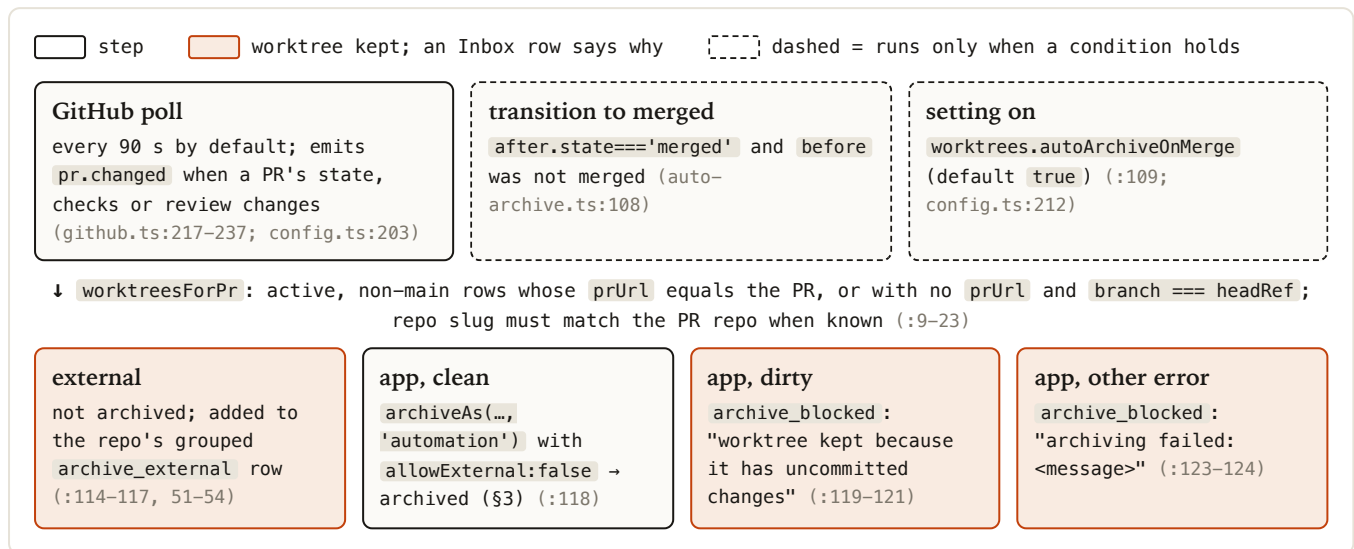
- If the folder is deleted outside the app, the next discovery marks the row `archived` and emits `worktree.removed`. No git command runs and no audit record is written. [worktree-read.ts:252-256]

Can it be undone?

- No restore route exists in `routes/worktrees.ts`, and the page cannot show archived rows. [routes/worktrees.ts:37-146; WorktreesPage.tsx:51]
- The branch survives, so the work is recoverable by hand: `git worktree add <path> <branch>`, then Discover. Discovery rewrites the row with `state='active'`, keeps its `createdByApp` and `createdAt`. [worktree-read.ts:218-242]
- New worktree with the same type, ticket and slug also works: the branch already exists, so it runs `git worktree add <path> <branch>` without `-b`. [worktree-write.ts:91-95, 117]
- Ignored files and revert refs are not recoverable. *(inferred from the steps above)*

4. Auto-archive on merge

Registered at daemon start by `registerAutoArchive(ctx)`. [daemon/main.ts:62; daemon/services/worktree/auto-archive.ts:31-135]



- **No retry.** Auto-archive fires only on the transition to merged. Cleaning a dirty worktree afterwards does not trigger a new attempt; the user archives it from the Worktrees tab, which resolves the row. [auto-archive.ts:107-129]

The two Inbox row shapes

	archive_blocked (one per worktree)	archive_external (one per repo)
For	App-created worktree that archive refused or failed on	Every merged worktree the app did not create, in one repo
Kind · key	<code>pr_event</code> , scope <code>{domain:'worktree', id:path}</code> , facet <code>archive_blocked</code> [auto-archive.ts:32-42]	<code>pr_event</code> , scope <code>{domain:'repo', id:repoPath}</code> , facet <code>archive_external</code> [archive-rows.ts:8, 15-16]
Reason text	<code><owner/repo>#<n></code> merged; worktree kept because <code><why></code> [auto-archive.ts:40]	<code>N</code> merged worktrees in <code><owner/repo></code> were not created by the app – archive them yourself [archive-rows.ts:18-21]
Payload	<code>{pr, event, path, presetId:null, vars:{}}</code> ; session, project and ticket copied from the worktree [auto-archive.ts:37-41]	<code>{event, repo, label, worktrees:[{path, pr}], ...}</code> , sorted by path [archive-rows.ts:51-58]
Inbox detail panel	"Open PR #n" button, and "Review" when a session is linked [web/features/inbox/PrEventActions.tsx:26-43]	List of paths, each with a link to its PR [PrEventActions.tsx:25, 57-77]

How the rows resolve

Besides the user marking a row Done in the Inbox, four paths auto-resolve it:

1. **The worktree is archived** (any caller) or its folder vanishes: `worktree.removed` → the per-worktree row is auto-resolved and the path is dropped from its repo's grouped row. [auto-archive.ts:56-59, 105]
2. **A grouped row loses its last path** → it is auto-resolved; otherwise its reason and count are rewritten. [archive-rows.ts:40-60, 77-86]
3. **The PR link changes.** When discovery finds the worktree now links a different PR than the row names, both kinds are resolved for that path. [worktree-read.ts:150-166, 244]
4. **Daemon start sweep.** Rows for inactive paths and rows naming a PR from another repo are resolved; old per-worktree "not created by the app" rows fold into the grouped row. [auto-archive.ts:61-103]

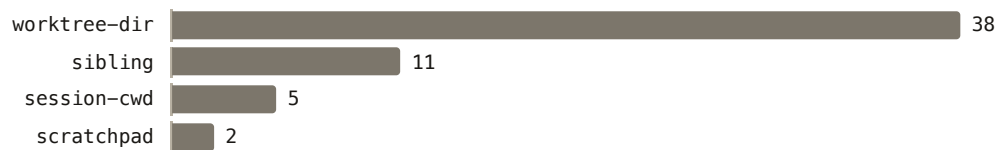
5. Live data from the running daemon

Read-only `GET /api/worktrees` and `GET /api/inbox` (no filters) against `127.0.0.1:4317` at `2026-09-30T10:12:27Z`. The `worktrees` table holds **111** rows.

Rows	main checkout	app-created	external	dirty	total
active	27	0	56	4	83
archived	0	0	28	0	28
total	27	0	84	4	111

bar length = number of active non-main worktrees; value printed at the bar's end

Active external worktrees by discovery origin (56)



Their PR status (56)



- **No app-created worktrees exist.** Every one of the 111 rows has `createdByApp=false`, so auto-archive has nothing it is allowed to archive in this data.
- **Active, other facts:** 27 repos; 42 of the 56 external worktrees carry a ticket; 17 have at least one linked session; 2 of them are dirty (the other 2 dirty rows are main checkouts).
- **Archived:** 28 rows, `archivedAt` between `2026-09-28T17:39:01Z` and `2026-09-30T10:04:25Z` (read from the `worktrees` table, since the API view drops `archivedAt` [`worktree-read.ts:70`]). The `audit_log` table has no `worktree.*` records, so these rows match the discovery path "folder gone → archived" rather than an Archive click (*inferred*).

Inbox rows raised by auto-archive

Row	open	auto_resolved	Detail
archive_external	4	1	41 worktrees across 4 repos: wakecap/infrastructure 36, wakecap/wakecap-wecare-service 3, wakecap/Wakecore 1, wakecap/frontend-2.0-om 1
archive_blocked	0	57	None open

- 41 of the 45 merged external worktrees are listed in a grouped row. The other 4 are all in `frontend-2.0-om`. The row is raised only when a `pr.changed` event carries the move into `merged`; a PR polled for the first time already merged still counts, because `before` is null [`auto-archive.ts:108`; `github.ts:233-236`]. So the likely cause is that these PRs were cached as merged before the worktree was linked to them, and no later change fired (*inferred, not traced*).

6. Declarations

packages/core/src/types/work.ts:26–36, 68–75, 88–96

```
export interface Worktree {
  path: string;
  repo: string;
  branch: string;
  base: string | null;
  ticket: string | null;
  dirty: boolean;
  prUrl: string | null;
  state: 'active' | 'archived';
  createdByApp: boolean;
}

export type WorktreeOrigin =
  | 'app'
  | 'config'
  | 'session-cwd'
  | 'claude-json'
  | 'worktree-dir'
  | 'sibling'
  | 'scratchpad';

export interface WorktreeView extends Worktree {
  head: string | null;
  isMain: boolean;
  origin: WorktreeOrigin;
  sessionPks: string[];
  projectId: string | null;
  prStatus: PrStatus | null;
  updatedAt: string;
}
```

apps/daemon/src/db/schema.ts:243–267

```
export const worktrees = sqliteTable(
  'worktrees',
  {
    path: text('path').primaryKey(),
    repo: text('repo').notNull(),
    branch: text('branch').notNull(),
    base: text('base'),
    ticket: text('ticket'),
    dirty: integer('dirty', { mode: 'boolean' }).notNull().default(false),
    prUrl: text('pr_url'),
    state: text('state', { enum: ['active', 'archived'] })
      .notNull()
      .default('active'),
    createdByApp: integer('created_by_app', { mode: 'boolean' }).notNull().default(false),
    head: text('head'),
    isMain: integer('is_main', { mode: 'boolean' }).notNull().default(false),
    origin: text('origin').notNull(),
    sessionPksJson: text('session_pks_json').notNull().default('[]'),
    projectId: text('project_id'),
    createdAt: text('created_at').notNull(),
    updatedAt: text('updated_at').notNull(),
    archivedAt: text('archived_at'),
  },
  (t) => [index('worktrees_repo_idx').on(t.repo), index('worktrees_branch_idx').on(t.branch)],
);
```

apps/daemon/src/services/worktree/worktree-sync.ts:87-112

```
const row = requireRow(d, path);
if (!row.createdByApp && !opts.allowExternal) {
  throw new GitError(
    'external_worktree',
    `${path} was not created by the app; confirm explicitly to archive it`,
  );
}
const reverts = revertRefPrefix(path);
if (existsSync(path)) {
  const dirty = await dirtyFiles(path);
  if (dirty.length > 0)
    throw new GitError(
      'dirty_worktree',
      `uncommitted changes in ${dirty.length} file(s): ${dirty.slice(0, 5).join(', ')}`,
    );
  const script = repoConfigFor(ctx, row.repo)?.archive;
  if (script)
    await execa(process.env.SHELL ?? '/bin/zsh', ['-lc', script], { cwd: path, timeout: 300_000 });
  await git(row.repo, ['worktree', 'remove', path]);
} else {
  await git(row.repo, ['worktree', 'prune']);
}
await pruneRevertRefs(row.repo, reverts);
const nowIso = d.now().toISOString();
markWorktreeArchived(ctx.db, path, nowIso);
ctx.bus.emit({ type: 'worktree.removed', path });
```

apps/daemon/src/services/git/exec.ts:45-50

```
if (cmd === 'worktree' && rest[0] === 'remove' && (rest.includes('--force') || rest.includes('-f')))
  deny('forced worktree removal');
if (cmd === 'checkout' && rest.includes('--'))
  deny('checkout of paths discards changes; use restore with a checkpoint');
if (cmd === 'branch' && (rest.includes('-D') || rest.includes('--delete') || rest.includes('-d')))
  deny('branch deletion');
```

packages/api-contract/src/config.ts:210-217

```
worktrees: z
  .object({
    autoArchiveOnMerge: z.boolean().default(true),
    scratchpadRoots: z.array(z.string()).default(['/private/tmp']),
    scanSiblings: z.boolean().default(true),
    implementTicketMode: z.enum(['precreate', 'conductor']).default('conductor'),
    checkpointsPerSession: z.number().int().positive().default(200),
  })
```

packages/api-contract/src/routes/worktrees.ts:26-30, 60-64

```
export const WorktreelistQuery = z.object({
  projectId: z.string().optional(),
  state: z.enum(['active', 'archived']).optional(),
  repo: z.string().optional(),
});

export const WorktreeArchiveBody = z.object({
  path: z.string().min(1),
  confirm: Confirm,
  confirmExternal: z.boolean().default(false),
});
```

7. What the code does not settle

- **Archive script failure.** The script runs through `execa` without `reject:false` [worktree-sync.ts:104], so a non-zero exit or the 300 s timeout throws before `git worktree remove` and the folder stays (*inferred: execa's default is to reject*). Any side effects the script already had are not rolled back.
- **Locked worktrees and submodules.** `git worktree remove` without `--force` refuses these, which would surface as `git_failed` (*inferred: git behaviour*). Nothing in the repo handles that case specially.
- **implementTicketMode.** The setting (`precreate` | `conductor`) exists in the config schema [config.ts:215], but nothing under `apps/daemon/src` or `apps/web/src` reads it [grep]. Whether a "precreate" flow is planned can't be told from the code.
- **Re-activated archived rows.** Discovery rewrites any active path it finds, including one that was archived, and keeps `createdByApp` [worktree-read.ts:218-242]. There is no record on the row that it was once archived, because `archivedAt` is reset to `null`.
- **Filtered lists after a live frame.** `worktree.updated` appends the row to every cached list, including lists for another project, until the next refetch. The code comment accepts this. [web/api/live-events.ts:58]
- **The 4 unlisted merged worktrees** in \$5 have no `archive_external` entry; the cause is inferred, not traced.